



Automating the Pipeline

AWS Academy Data Engineering

Introduction

Automating the Pipeline

Module objectives

This module prepares you to do the following:

- Identify the benefits of automating your pipeline.
- Understand the role of CI/CD and how it applies to your pipeline.
- Examine the states of AWS Step Functions.
- Use Step Functions to build and automate a data pipeline.

Module overview

Presentation sections

- Automating infrastructure deployment
- CI/CD
- Automating with Step Functions

Lab

- Building and Orchestrating ETL Pipelines by Using Athena and Step Functions

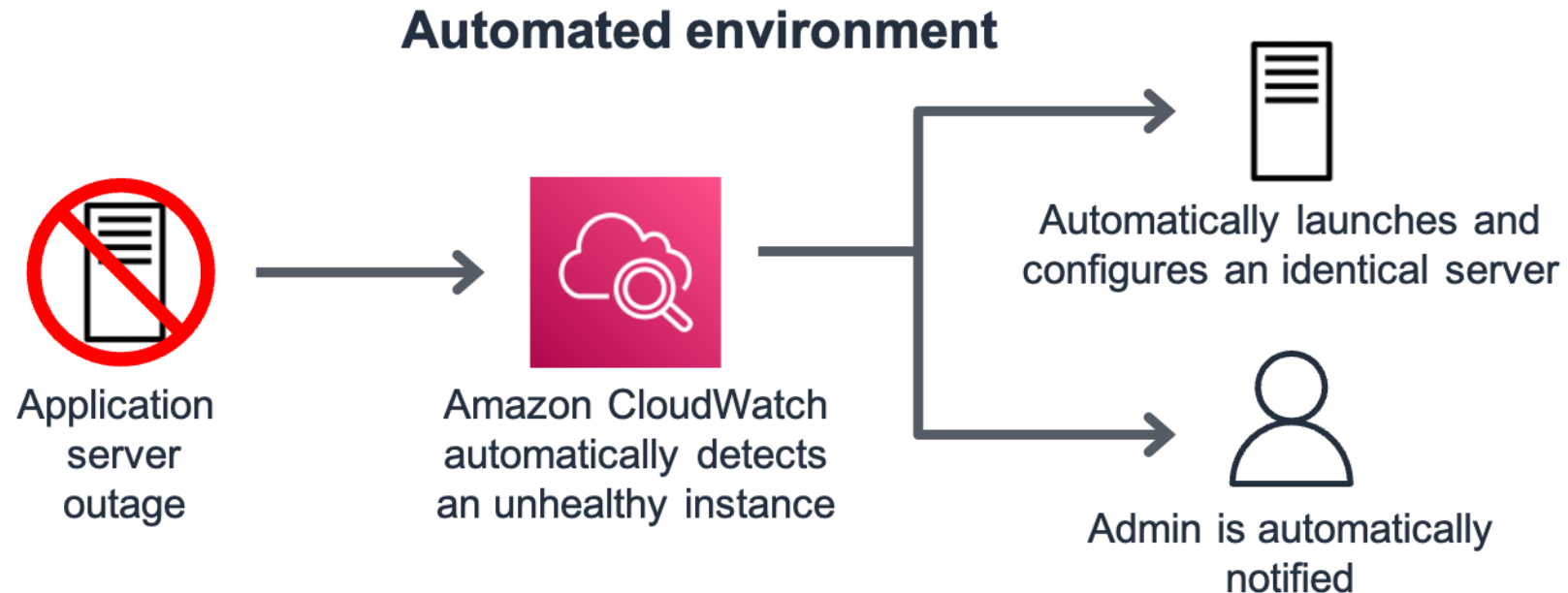
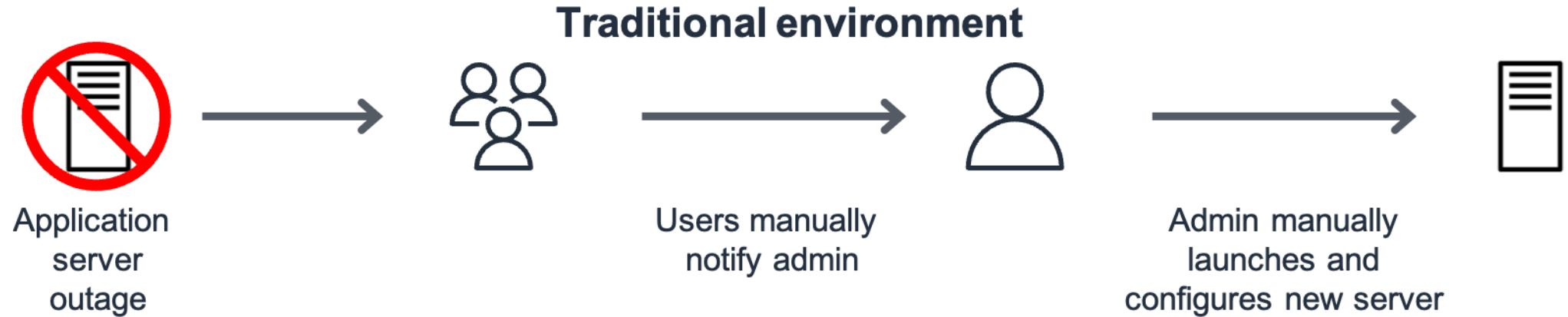
Knowledge checks

- Online knowledge check
- Sample exam question

Automating infrastructure deployment

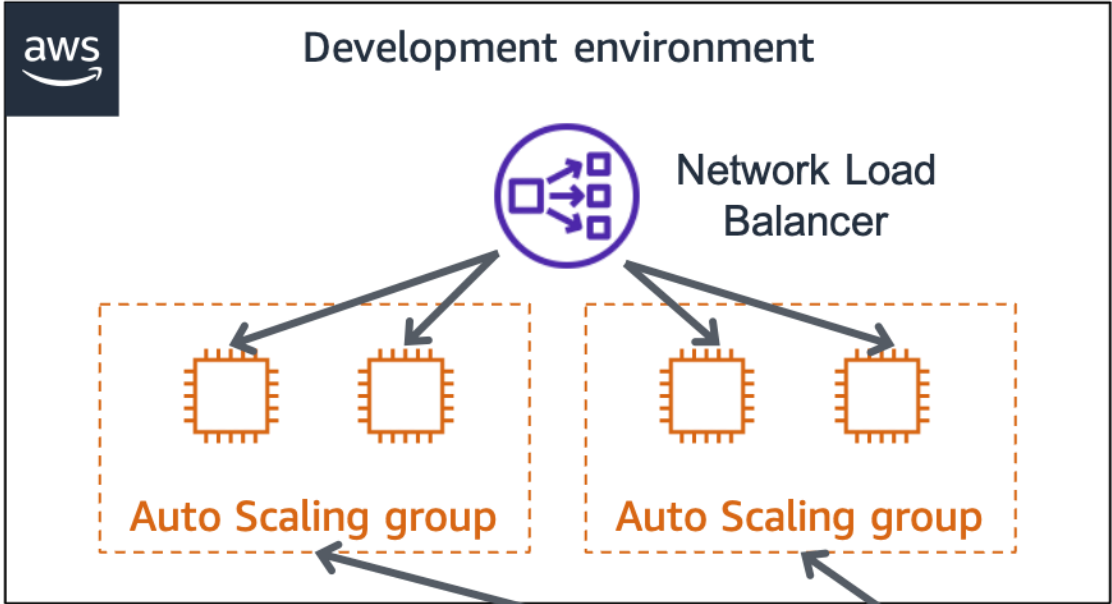
Automating the Pipeline

Automate your environment

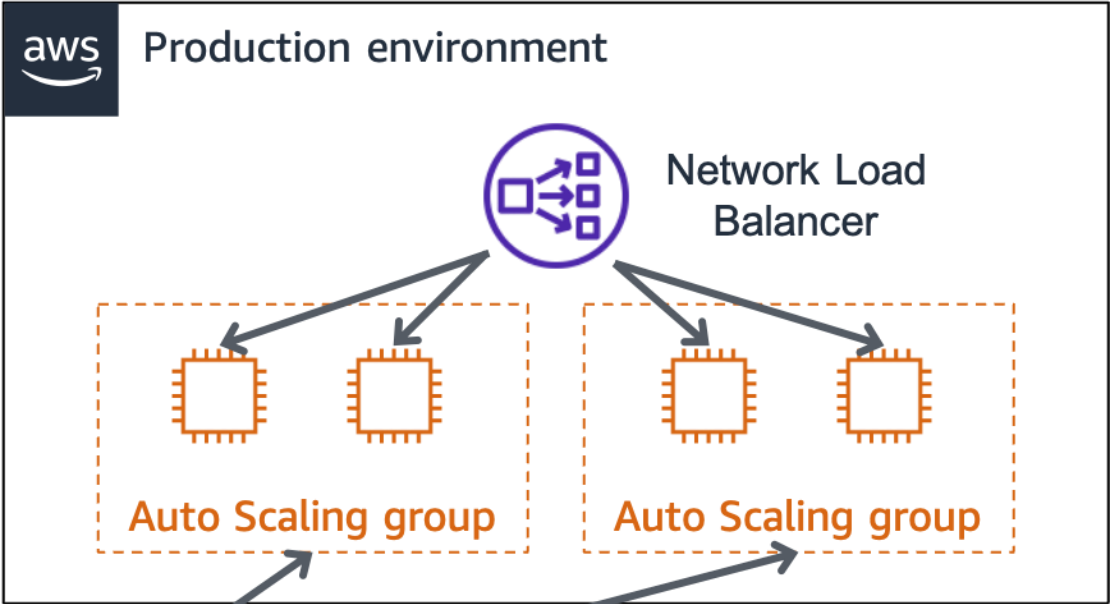


Infrastructure as code

Repeatability



Reusability



Template

Key takeaways: Automating infrastructure deployment

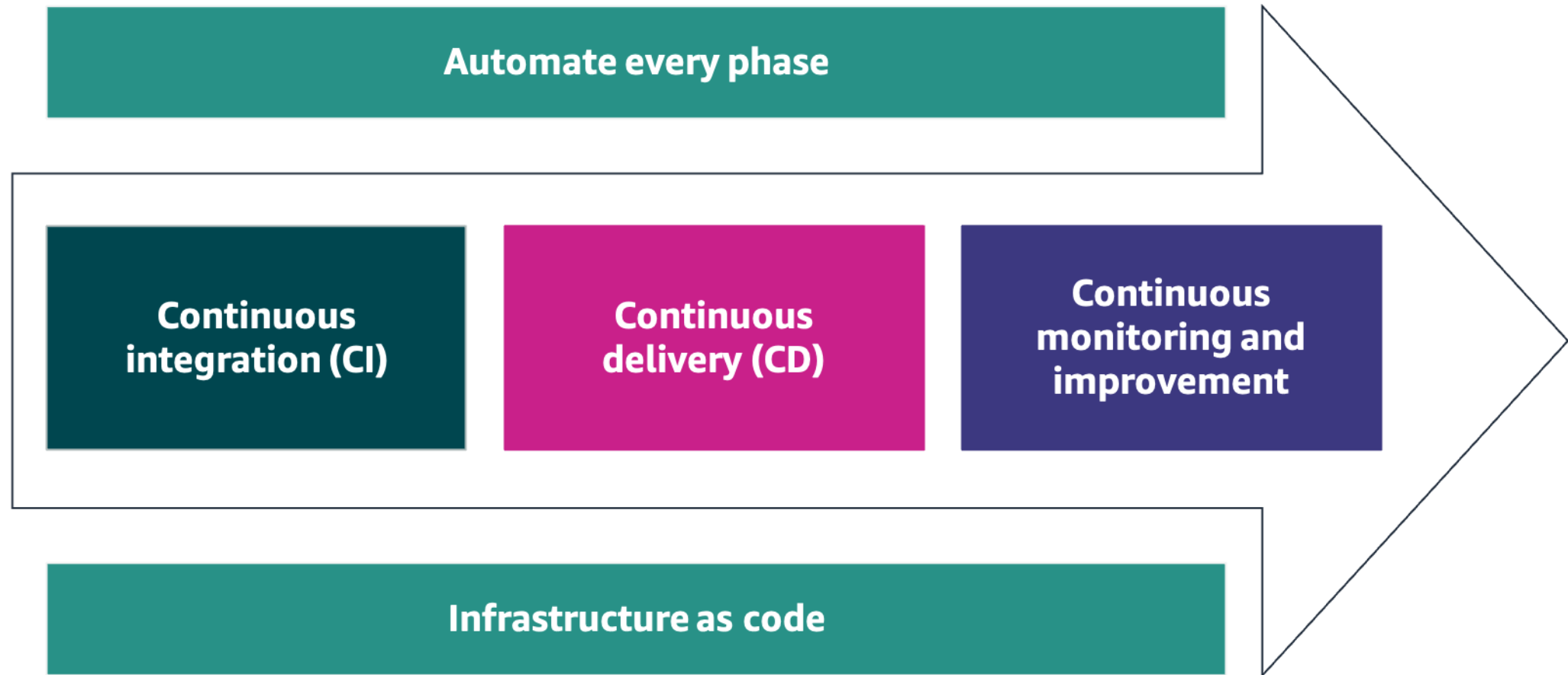


- Automating your environment can help you ensure that your system is stable, consistent, and efficient.
- Repeatability and reusability are two key benefits of infrastructure as code.

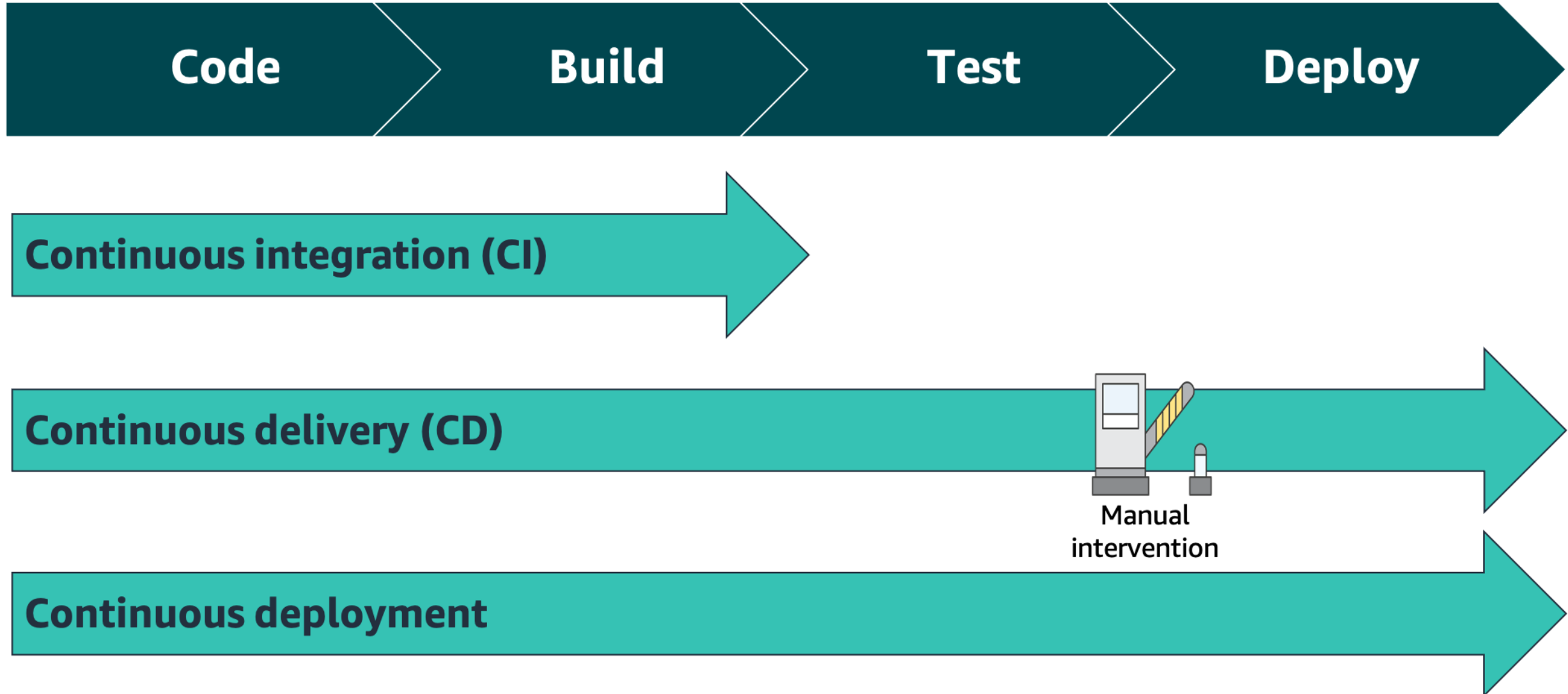
CI/CD

Automating the Pipeline

CI/CD as part of DevOps



Understanding CI/CD



Key takeaways: CI/CD



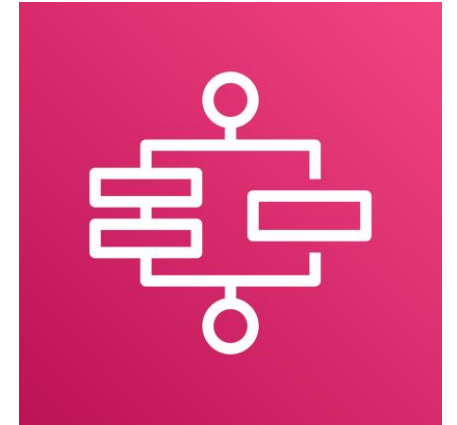
- CI/CD spans the develop and deploy stages of the software development lifecycle.
- Continuous delivery improves on continuous integration by helping teams gain a greater level of certainty that their software will work in production.

Automating with Step Functions

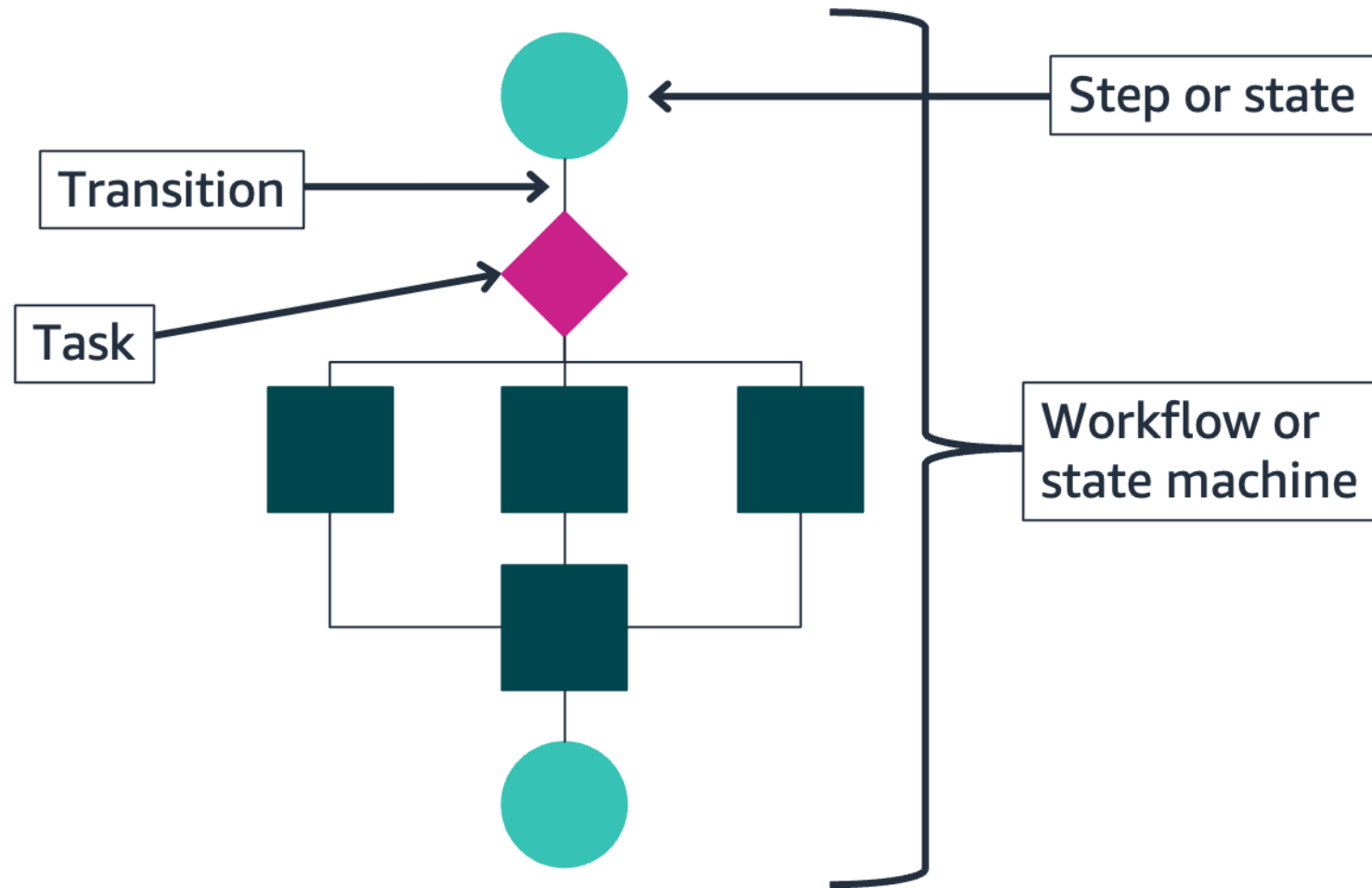
Automating the Pipeline

AWS Step Functions

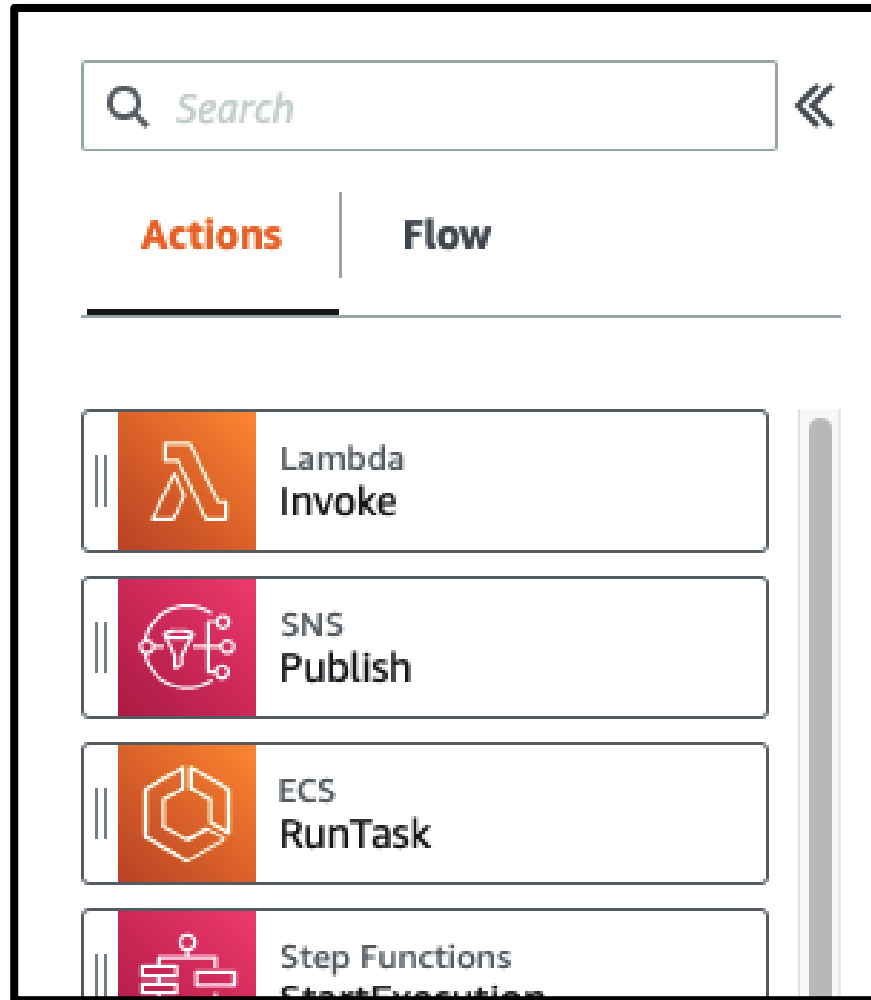
- Uses visual workflows to coordinate components of distributed applications and microservices
- Automatically initiates and tracks each step, and retries when errors occur
- Logs the state of each step
- Integrates with Amazon Athena



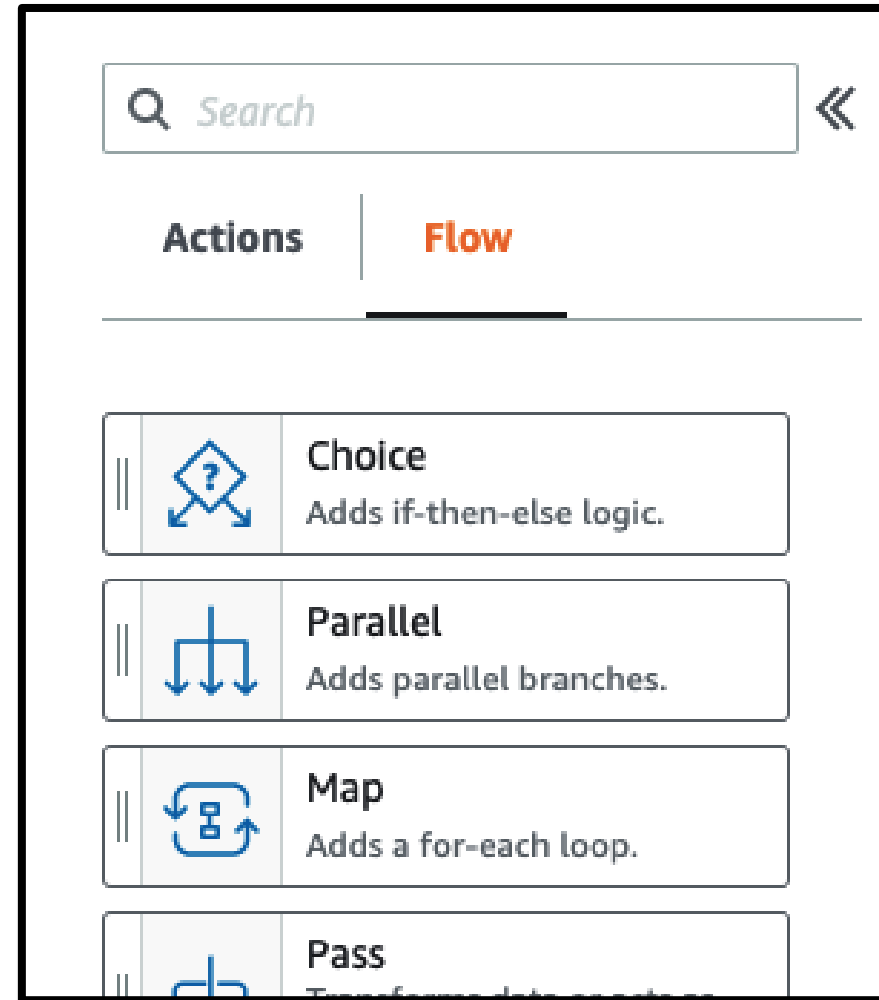
How Step Functions works



Step Functions interface: States browser



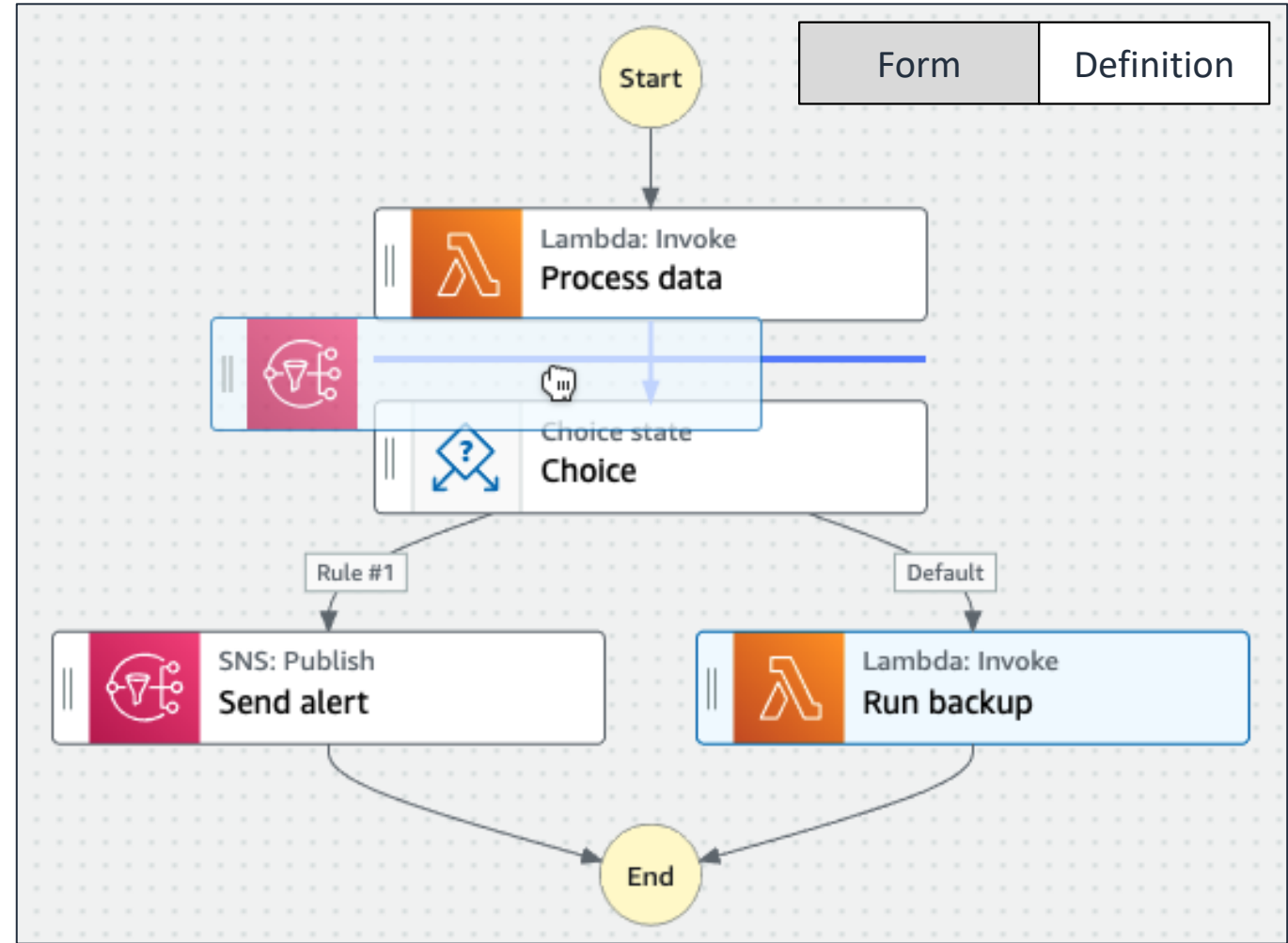
Actions panel



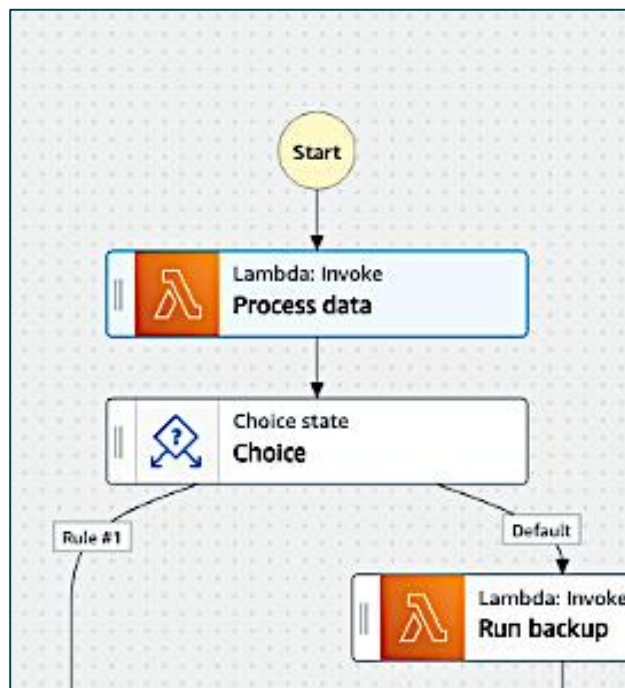
Flow panel

Step Functions interface: Canvas

- From the **Actions** and **Flow** panels, drag and drop states to the canvas.
- When a new state is added to your workflow, the code for that state is generated automatically.



Step Functions interface: Inspector



```
"Process data": {
  "Type": "Task",
  "Resource": "arn:aws:states:::lambda:invoke",
  "OutputPath": "$.Payload",
  "Parameters": {
    "Payload.$": "$"
  },
  "Retry": [
    {
      "ErrorEquals": [
        "Lambda.ServiceException",
        "Lambda.AWSLambdaException",
        "Lambda.SdkClientException"
      ],
      "IntervalSeconds": 2,
      "MaxAttempts": 6,
      "BackoffRate": 2
    }
  ],
  "Next": "Choice"
},
"Choice": {
  "Type": "Choice",
  "Choices": [
    {
      "Next": "Send alert"
    }
  ]
}
```

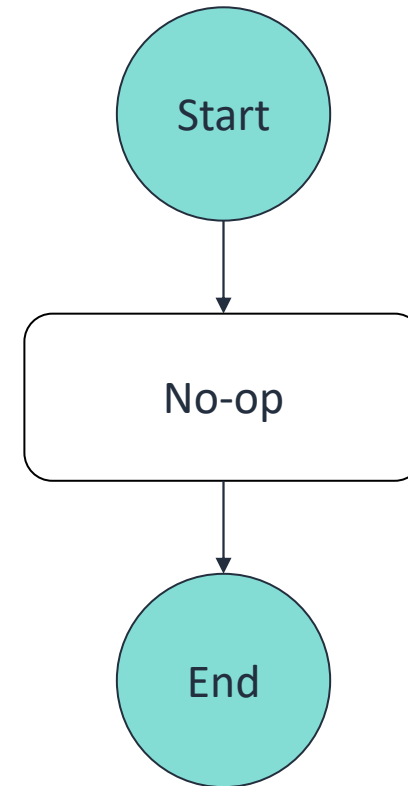
Form	Definition
Process data	
Configuration Input Output Error handling	
State name Process data	
API Lambda: Invoke Info	
Function name The Lambda function to invoke Choose an option ▼	
Payload The JSON that you want to provide to your Lambda function. Use state input as payload ▼	

State types

Task	Single unit of work performed by a state machine
Pass	Passes input to output, without performing work
Choice	Adds branching logic to a state machine
Parallel	Creates parallel branches for running states
Wait	Delays from continuing for a specified
Map	Dynamically iterates steps
Succeed	Stops running a state successfully
Fail	Stops running a state and marks it as a failure

Pass state

```
"No-op": {  
  "Type": "Pass",  
  "Result": {  
    "x-datum": 0.381018,  
    "y-datum": 622.2269926397355  
  },  
  "ResultPath": "$.coords",  
  "Next": "End"  
}
```

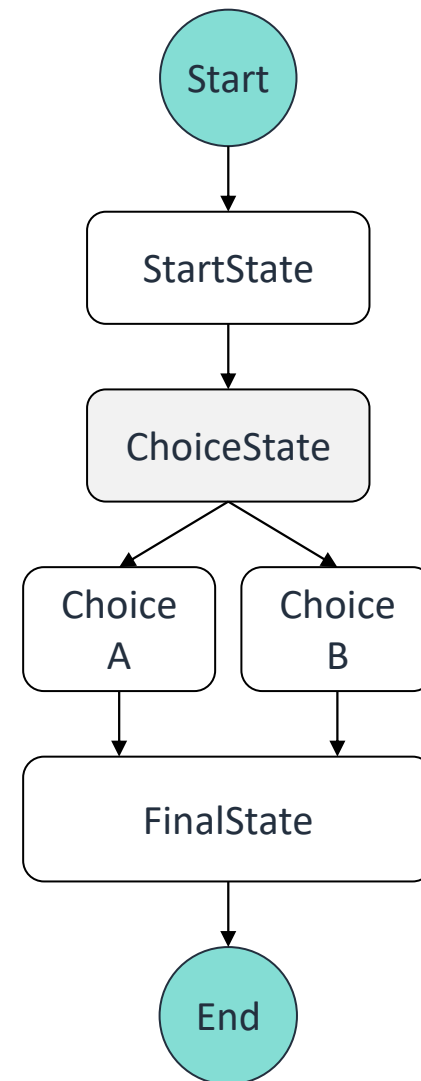


Choice state

```
"ChoiceState": {  
  "Type" : "Choice",  
  "Choices": [  
    {  
      "variable": "$.foo",  
      "NumericEquals": 1,  
      "Next": "Choice A"  
    },  
    {  
      "variable": "$.foo",  
      "NumericEquals": 2,  
      "Next": "Choice B"  
    }  
  ],  
  "Default": "DefaultState"  
}
```

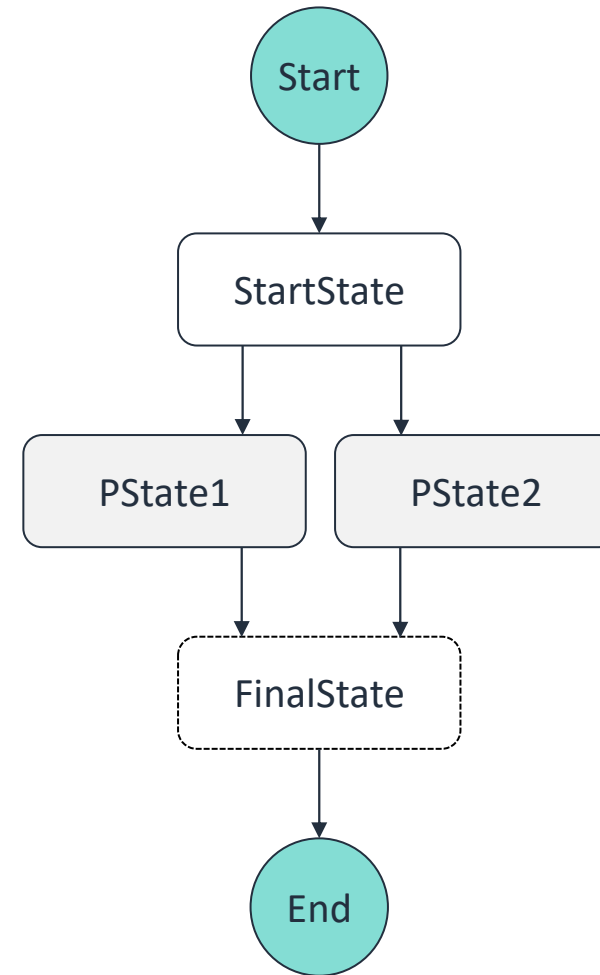
Choice rule

Choice rule



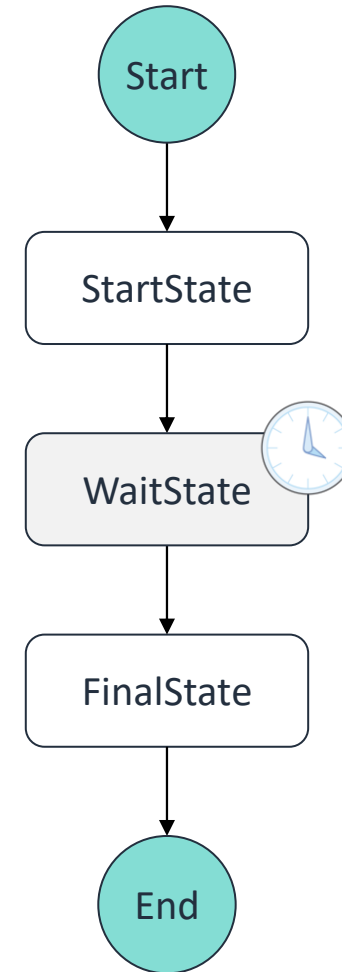
Parallel state

```
"LookupCustomerInfo": {
  "Type": "Parallel",
  "Branches": [
    {
      "StartAt": "PState1",
      "States": {
        "PState1": {...}
      }
    },
    {
      "StartAt": "PState2",
      "States": {
        "PState2": {...}
      }
    }
  ],
  "Next": "NextState"
}
```



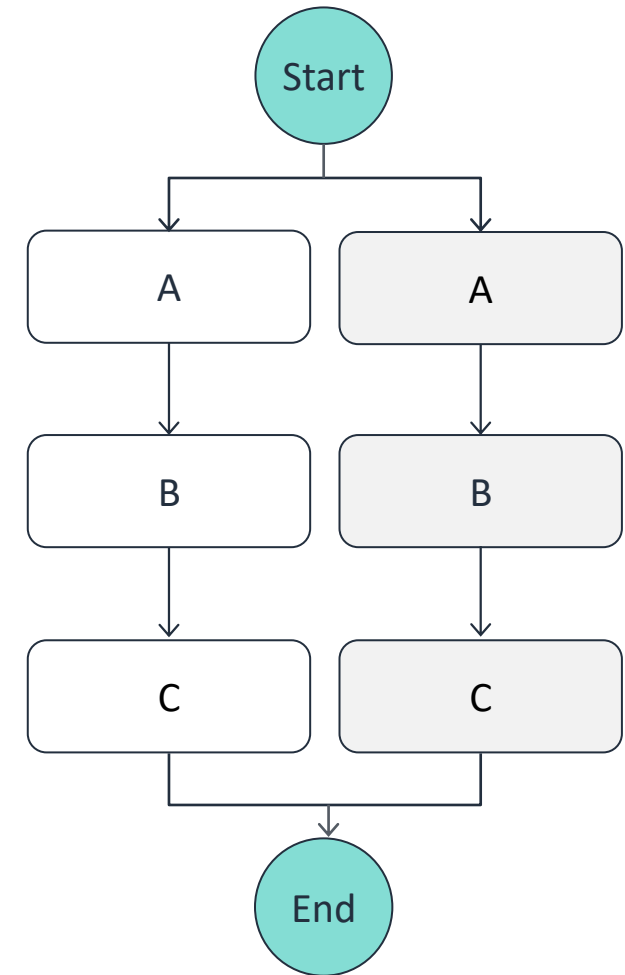
Wait state

```
"wait_ten_seconds": {  
  "Type": "wait",  
  "Seconds": 10,  
  "Next": "NextState"  
}
```



Map state

```
"validate-All": {  
  "Type": "Map",  
  "InputPath": "$.detail",  
  "ItemsPath": "$.shipped",  
  "MaxConcurrency": 0,  
  "Iterator": {  
    "StartAt": "validate",  
    "States": {  
      "validate": {  
        "Type": "Task",  
        "Resource": "arn:aws:lambda:us-east-1:123456789012:function:ship-val",  
        "End": true  
      }  
    }  
  }  
}
```



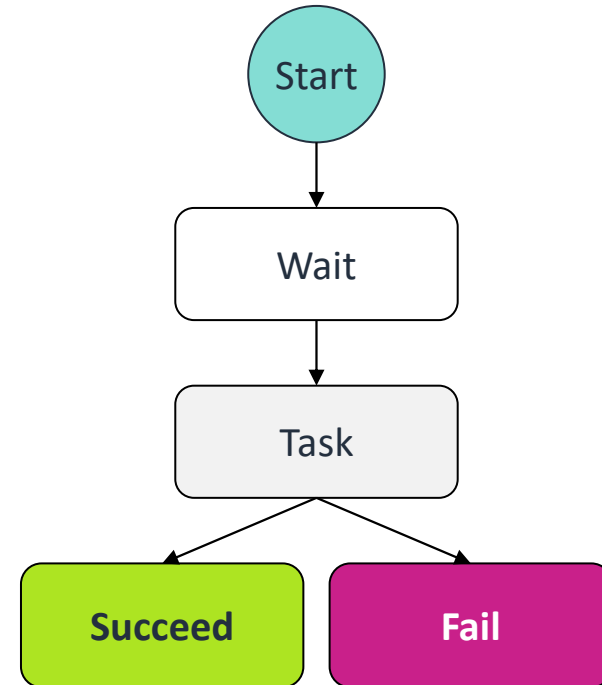
Succeed and Fail states

Succeed state

```
"SuccessState": {  
  "Type": "Succeed"  
}
```

Fail state

```
"FailState": {  
  "Type": "Fail",  
  "Cause": "Invalid response.",  
  "Error": "ErrorA"  
}
```



Simplifying ETL pipelines with Step Functions



Key takeaways: Automating with Step Functions



- With Step Functions, you can use visual workflows to coordinate the components of distributed applications and microservices.
- You define a *workflow*, which is also referred to as a *state machine*, as a series of steps and transitions between each step.
- Step Functions is integrated with Athena to facilitate building workflows that include Athena queries and data processing operations.

Lab: Building and Orchestrating ETL Pipelines by Using Athena and Step Functions

Lab introduction: Building and Orchestrating ETL Pipelines by Using Athena and Step Functions



- In this lab, you will use Step Functions to build an ETL pipeline that uses Amazon S3, an AWS Glue Data Catalog, and Athena to process a large dataset.
- You will design the workflow so that if AWS Glue tables don't exist, the workflow will invoke additional Athena queries to create them.
- Open your lab environment to start the lab and find additional details about the tasks that you will perform during this lab.

Debrief: Building and Orchestrating ETL Pipelines by Using Athena and Step Functions

- Why did you need to configure routing logic in the Step Functions workflow?
- When and how did you implement Snappy compression and ensure that the data would be stored in Parquet format? What is the benefit of storing the data this way?
- What is the purpose of creating a state machine in a service such as Step Functions?

Module wrap-up

Automating the Pipeline

Module summary

This module prepared you to do the following:

- Identify the benefits of automating your pipeline.
- Understand the role of CI/CD and how it applies to your pipeline.
- Examine the states of Step Functions.
- Use Step Functions to build and automate a data pipeline.

Module knowledge check



- The knowledge check is delivered online within your course.
- The knowledge check includes 10 questions based on material presented on the slides and in the slide notes.
- You can retake the knowledge check as many times as you like.

Sample exam question

A data engineer is using Step Functions to automate a pipeline to handle an analytics workload. They are using the Workflow Studio interface to drag and drop components into the workflow. However, the engineer isn't sure if the auto-generated code will perform the tasks and processing that are required. Which element of the Step Functions interface can they use to examine the code for accuracy?

Identify the key words and phrases before continuing.

The following are the key words and phrases:

- **Step Functions**
- **Auto-generated code**
- **Examine the code**

Sample exam question: Response choices

A data engineer is using **Step Functions** to automate a pipeline to handle an analytics workload. They are using the Workflow Studio interface to drag and drop components into the workflow. However, the engineer isn't sure if the **auto-generated code** will perform the tasks and processing that are required. Which element of the Step Functions interface can they use to **examine the code** for accuracy?

Choice	Response
A	Use the States browser to examine the code.
B	Use the Form panel to view the code.
C	Review the definition area on the Inspector panel.
D	Run the Step Functions workflow and examine the output.

Sample exam question: Answer

The correct answer is C.

Choice	Response
--------	----------

C	Review the definition area on the Inspector panel.
---	---

Thank you



Corrections, feedback, or other questions?

Contact us at <https://support.aws.amazon.com/#/contacts/aws-academy>.

All trademarks are the property of their owners.